

Projeto técnico: Sistema de emissão de Notas Fiscais

Objetivo

Desenvolver uma aplicação **em Angular**, conforme os requisitos descritos abaixo, e apresentar os resultados **em formato de vídeo**, demonstrando:

- As **telas desenvolvidas**;
- As **funcionalidades implementadas**;
- Um **detalhamento técnico** da solução.

No detalhamento técnico, informar:

- Quais **ciclos de vida do Angular** foram utilizados;
- Se foi feito uso da biblioteca **RxJS** e, em caso afirmativo, como;
- Quais **outras bibliotecas** foram utilizadas e para qual finalidade;
- Para **componentes visuais**, quais bibliotecas foram utilizadas;
- Como foi realizado o **gerenciamento de dependências** no **Golang** (se aplicável);
- Quais **frameworks** foram utilizados no **Golang** ou **C#**;
- Como foram tratados os **erros e exceções** no backend;
- Caso a implementação utilize **C#**, indicar se foi utilizado **LINQ** e de que forma.

Escopo

1. Funcionalidades a serem desenvolvidas

Cadastro de Produtos

Campos obrigatórios:

- Código
- Descrição (nome do produto)
- Saldo (quantidade disponível em estoque)

Resultado esperado: permitir que um produto seja previamente cadastrado para posterior utilização em notas fiscais.

Cadastro de Notas Fiscais

Campos obrigatórios:

- Numeração sequencial
- Status: *Aberta* ou *Fechada*

- Inclusão de múltiplos produtos com respectivas quantidades

Resultado esperado: permitir a criação de uma nota fiscal com numeração sequencial e status inicial *Aberta*.

Impressão de Notas Fiscais

- Botão de impressão visível e intuitivo em tela.

Resultado esperado:

- Ao clicar no botão, exibir indicador de processamento;
- Após finalização, atualizar o status da nota para *Fechada*;
- Não permitir a impressão de notas com status diferente de *Aberta*;
- Atualizar o saldo dos produtos conforme a quantidade utilizada na nota.

○ Exemplo: saldo anterior = 10; nota utiliza 2 unidades → novo saldo = 8.

Requisitos obrigatórios

1. Arquitetura de Microserviços:

Estruturar o sistema com **no mínimo dois microserviços**:

- **Serviço de Estoque** – controle de produtos e saldos;
- **Serviço de Faturamento** – gestão de notas fiscais.

2. Tratamento de Falhas:

Implementar um cenário em que **um dos microserviços falha**.

O sistema deve ser capaz de **se recuperar** da falha e **fornecer feedback apropriado ao usuário** sobre o erro.

3. Conexão Real com banco de dados:

É esperado que os cadastros sejam persistidos fisicamente em um banco de dados de sua escolha.

Requisitos opcionais

O candidato poderá, a seu critério, implementar também:

a. Tratamento de Concorrência:

Cenário: produto com saldo 1 sendo utilizado simultaneamente por duas notas.

b. Uso de Inteligência Artificial:

Implementar alguma funcionalidade do sistema que utilize IA.

c. Implementação de Idempotência:

Garantir que operações repetidas não causem efeitos colaterais indesejados.

Orientações para entrega

1. O projeto deverá ser disponibilizado em um repositório público do GitHub com o nome: **Korp_Teste_SeuNome**
2. Após finalizar o desenvolvimento, o candidato deve enviar **por e-mail**, em até **7 dias corridos** após o recebimento deste desafio, o material concluído. (Envie para rh@korp.com.br)
3. O envio deve conter obrigatoriamente:
 - Link público do repositório GitHub **Korp_Teste_SeuNome**
 - Link para vídeo de apresentação das telas e funcionalidades implementadas (Google Drive, One Drive ou alguma outra nuvem da sua preferência)
 - Link do arquivo de detalhamento técnico conforme itens descritos no documento de especificação deste teste